EFE — Deux audits de code ancrés

La chaîne d'audit HMAC et l'ordonnanceur DRR (N15)

Crate `efe-nano` (runtime M50) — lecture directe des sources + vérification adverse — 19 juillet 2026

`efe-nano` est une **bibliothèque** d'exécution d'agents (`[lib]`, aucun `[[bin]]`), réelle et lourdement testée mais **dormante** : aucun binaire de production ne construit son dispatcher ni ses runtimes. Ce document isole ses **deux sous-systèmes les plus sérieux** et les audite au niveau du code. La chaîne HMAC est le seul actif qui *essaime* : `efe-audit-core` et `efe-swarm-provenance` se décrivent comme des répliques exactes de sa construction. L'ordonnanceur DRR est le morceau algorithmique le plus falsifiable (équité, anti-famine, conservation du travail, concurrence).

Méthode. Chaque affirmation est ancrée `fichier:ligne` et provient d'une lecture directe des sources, recoupée par une passe adverse multi-agents (16 lecteurs/chercheurs, chaque observation crypto et chaque *finding* vérifié par réfutation). Les constats sont gradués **ÉLEVÉ** / **MOYEN** / **FAIBLE**. On distingue partout le **réel/mesuré** du **documenté/stub**.

1 La chaîne d'audit HMAC — le reçu re-vérifiable

1.1 Les cinq primitives

Primitive	Formule réelle	Ancre
Corps canonique	`canonical_row_body = canonicalize_json({≈18 champs, clés triées, sans espaces})` ; exclut `full_input/output, tombstone, payload_hash, hmac, chain_parent_id, inserted_at`	`payload.rs:44`
Hash de charge	`payload_hash = SHA-256(corps)` — SHA-256 **nu, non clavé**, 32 o	`payload.rs:160`
Sceau de chaîne	`hmac = HMAC-SHA256(parent||payload_hash, tenant_key)` où `parent` = UUID 16 o **ou** `b"genesis"` 7 o	`payload.rs:177`
Clé par tenant	`HKDF-SHA256(IKM=master, salt=tenant_uuid[16], info=b"efe-nano/audit/v1", L=32)`	`hkdf.rs:99`
Sceau de tombale	`HMAC-SHA256(b"tombstone-v1"||trace_id||...||actor, tenant_key)` (effacement RGPD)	`payload.rs:219`

Le chaînage (`hmac_chain.rs:135`) : `entry[n].chain_parent_id = entry[n-1].trace_id` ; la racine a `chain_parent_id = None`. **L'ordre n'est authentifié que par ce lien parent** — aucun numéro de séquence n'est scellé dans le corps.

1.2 Le chemin d'écriture : `append_chained` (`hmac_chain.rs:133`)

1. `parent = current_tip(tenant)` — tip lu d'un cache `Mutex<HashMap<TenantId,Uuid>>` (`:50`), sinon du store ; `None` ⇒ la ligne devient la genèse.
2. `key = tenant_key(tenant)` — dérivée à la volée puis mise en cache `Mutex<HashMap<TenantId,Vec<u8>>>` (`:53`), **en clair, jamais zeroïsée**.
3. `payload_hash = compute_payload_hash(&trace)` : SHA-256 du corps canonique. Le corps fige la déterminisme par champ — `cost_cents` en `format!("{:.6}")` (`payload.rs:63`), hashes en hexa, `parent_trace_ids` triés (`:125`), timestamps en `to_rfc3339()` (`:139`).
4. `hmac = compute_hmac(parent, &payload_hash, &key)` : absorbe `parent.as_bytes()` (16 o) **ou** `b"genesis"` (7 o), puis `payload_hash` (32 o), sans préfixe de longueur ni délimiteur (`payload.rs:186--189`).
5. `store.append(entry)` — écriture **à tentative unique**, sans retry (`:149`) ; puis avance le tip caché.

Le store applique lui-même le **contrôle de tip** : `expected_parent = tenant_chain.last().trace_id()` ; rejette si `chain_parent_id != expected_parent` (`chain_store.rs:340`), sous un **unique** `Mutex` **global** `HashMap<TenantId,Vec<ChainEntry>>` (`:264`). Deux `append` concurrents pour le *même* tenant ne peuvent donc pas forker — mais comme `record()` **avale l'erreur et laisse tomber la trace** (`hmac_chain.rs:180`), le perdant est **silencieusement perdu**. Aucun test ne couvre l'append concurrent même-tenant.

1.3 Le chemin de vérification : `verify_entry` (`verify.rs:151`)

Pour chaque entrée, dans l'ordre : **(1)** lien parent `if chain_parent_id != expected_parent -> ChainBroken` (`:157`) ; **(2)** recalcul `compute_payload_hash`, `if != payload_hash -> HmacMismatch` (`:162`) ; **(3)** recalcul `compute_hmac`, `if != hmac -> HmacMismatch` (`:168`). `verify_chain` amorce le parent attendu depuis le `chain_parent_id` de la **première ligne fournie elle-même** (`:77`), jamais confronté à la genèse ni à un tip épinglé.

1.4 Clé et couches d'extension

HKDF (`hkdf.rs:99`) : `salt` = UUID du tenant (public, prédictible) ; seul secret = le master (`EnvHkdf` ≥32 o, `:68` ; `StaticMaster` ≥ **16** o, `:77`) ; `info=b"efe-nano/audit/v1"` est le *seul* tag de version de tout le schéma, et il vit dans la dérivation de clé, pas dans le message HMAC.

Époque Merkle (`merkle/tree.rs`) : arbre **RFC-6962 correct** — `leaf = SHA-256(0x00||payload)` (`:103`), `node = SHA-256(0x01||L||R)` (`:114`) : la **défense anti-second-préimage** (tags 0x00/0x01) est bien implémentée. Racine d'époque signée `HMAC-SHA256(tenant_audit_key, canonical_root_body)` (`epoch.rs:364`), chaînée à l'époque précédente.

Notaire externe (`merkle/notary.rs`) : le seul impl est `InMemoryNotary`, qui **signe avec une clé HMAC dans le même processus**. L'en-tête du fichier nomme lui-même la menace (un opérateur EFE compromis pourrait re-signer) puis livre un impl qui **ne la contre pas** ; l'ancrage externe (Sigstore Rekor, Ed25519) est **Phase 4, non codé**, et le sceleur d'époque ne branche aucun notaire.

1.5 Constats — chaîne HMAC

ÉLEVÉ 1 — La chaîne seule ne détecte pas la troncature/rollback

`verify_chain` n'ancre ni la tête (vs genèse) ni le tip (vs valeur notariée) — `verify.rs:77`. Supprimer une entrée du *milieu* casse le lien parent et est détecté ; supprimer les N **dernières** produit une chaîne plus courte qui **vérifie parfaitement**. La parade (époque Merkle notariée) existe en code, mais son notaire est un HMAC intra-processus non branché, et le vrai notaire externe est Phase 4. `verify_epoch_chain` ne prouve donc qu'une **auto-cohérence interne** : un opérateur compromis qui altère une trace, recalcule les `payload_hash` en aval et re-signe passe tous les contrôles. **Garantie réelle : tamper-evident pour un détenteur de clé avec tip épinglé hors-bande — pas trustless, pas anti-opérateur tel que livré.**

ÉLEVÉ 2 — Le sceau de tombale est calculé mais jamais vérifié

`compute_tombstone_hmac` existe (`payload.rs:219`) et `TombstoneMetadata` le porte, mais *aucun* de `verify_chain / verify_subchain / verify_entry` ne le recalcule. De plus `tombstone` est **exclu du corps canonique** (`payload.rs:44`) — donc hors du HMAC de chaîne. Le reçu d'intégrité de l'effacement RGPD n'est rattaché à rien qui soit vérifié.

ÉLEVÉ 3 — La dédup globale de corps défait l'effacement RGPD

La purge par refcount (`chain_store.rs:471`) déduplique les corps par SHA-256 **globalement, tous tenants confondus**, et ne supprime un corps que si *aucune* trace non-tombée d'*aucun* tenant ne le référence encore. Tomber le tenant A **n'efface pas physiquement** sa PII si le tenant B détient un corps de hash identique. Les tombées par lot (`:531`) sont en outre *collect-then-act, non atomiques* (**MOYEN**).

MOYEN 4 — Comparaison non constant-time

`verify_entry` compare `payload_hash` et `hmac` avec un `!=` sur `[u8;32]` (`verify.rs:163,169`) — **pas** `subtle::ConstantTimeEq`, aucun `ct_eq` dans tout le sous-système. Impact pratique faible (vérification hors-ligne, pas d'oracle en ligne). *À corriger dans le discours* : si un `ct_eq` a été promis, il concerne le crate `provenance-dag` ; pour le vérificateur d'`efe-nano` l'affirmation serait fausse.

MOYEN 5 — La canonicalisation partagée n'échappe pas les clés d'objet

`write_canonical` échappe les *valeurs* chaîne caractère par caractère (`idempotency/mod.rs:466`) mais écrit les *clés* **brutes** : `out.push_str(k)` (`:501`). Une clé contenant `"` ou `\` injecte de la structure : deux `Value` distincts peuvent produire les **mêmes octets canoniques**. **Inatteignable dans la chaîne d'audit** (clés littérales figées), mais **réel dans le chemin d'idempotence** où les clés d'input sont contrôlées par l'appelant : confusion de cache.

FAIBLE 6 — Nits durcissables

Pas de tag de domaine sur le HMAC principal alors que la tombale en a un (`b"tombstone-v1"`), les deux sous la même clé (`payload.rs:185`) ; `StaticMaster` accepte 16 o quand `EnvHkdf` exige ≥32 (`hkdf.rs:77`) ; clé jamais zeroïsée (`hkdf.rs:105`) ; longueur de clé HMAC non validée, clé vide acceptée (`payload.rs:183`) ; doc "précision seconde" alors que `to_rfc3339()` émet le sous-seconde (`payload.rs:139`) ; `Mutex` *global* contredisant la doc "Mutex par tenant" (`chain_store.rs:264`).

Ce qui est bien fait. HKDF par tenant correcte (isolation cryptographique réelle) ; domaine séparé pour la tombale ; tags anti-second-préimage Merkle corrects (RFC-6962) ; **layout HMAC non ambigu par champs de largeur fixe** (16 ou 7, puis 32 — pas d'injection par délimiteur, et genèse vs parent également tranché par le contrôle de lien parent) ; `payload_hash` SHA-256 nu **sans exposition length-extension** car consommé uniquement à l'intérieur du HMAC.

2 L'ordonnanceur N15 (Deficit Round Robin)

2.1 Ce qui tient — la sûreté

Le DRR **ne sur-admet jamais** : `advance` octroie **au plus un** slot par appel (il `break` après le premier octroi, `drr.rs:697`) sous la garde `self.inflight >= capacity -> None` (`:639`) ; le `DrrPermit` libère par `Drop` (`:218`) ; l'invariant de capacité tient sous concurrence (test `:768`). Le token-bucket rejette proprement les coûts impossibles, refill paresseusement, gère NaN/inf (`token_bucket.rs:159`). **Les défauts sont de fairness et de liveness, pas de sûreté.**

2.2 Constats — DRR

ÉLEVÉ 1 — Le poids DRR est inerte : c'est du round-robin strict, pas de l'équité pondérée

À chaque visite : `deficit += weight` (drr.rs:661), qualifie si `deficit >= 1 || starved` (:665), et sur octroi `deficit -= 1` (:669). Or `weight` est borné à ≥1 (:591), donc après `+= weight` le déficit est **toujours ≥1** : **tout tenant qualifie dès sa première visite**, `advance` octroie et `break`. Le déficit d'un tenant lourd s'accumule **sans jamais être converti en service supplémentaire** (l'inner-loop classique du DRR "sert tant que déficit ≥ coût" est absent). Mesurable : A(w=1) et B(w=3) saturés reçoivent **50/50**, pas 25/75. **Le poids n'a strictement aucun effet ; la fonctionnalité de tête — l'équité pondérée — n'existe pas.**

ÉLEVÉ 2 — `acquire` n'est pas cancellation-safe : fuite d'état DRR → rétrécissement monotone → wedge

`acquire_with_tier` (drr.rs:409) ne nettoie (`cancel_pending`) que sur ses deux branches de timeout (:465,:493) ; **aucun garde Drop**. Les auteurs savaient pourtant gérer l'annulation : `admit_swarm` installe un `TrackerRollbackGuard` (G11, admission.rs:261) qui rembobine **les compteurs du tracker** sur drop — mais **pas l'état interne du pool DRR**. Si la future est **droppée** pendant le `.await` (déconnexion client, `select!`, timeout externe), le tracker est rembobiné mais l'inscription du waiter DRR (FIFO + `waiters`) n'est jamais nettoyée, et tout octroi déjà placé dans `outstanding_grants` (avec `self.inflight++`) **fuit** (aucun `DrrPermit` construit → aucun `Drop`). Chaque acquire annulée laisse fuir **un slot inflight permanent** ; après `capacity` annulations, le pool est **coincé en backpressure permanente**. Le garde G11 a colmaté une couche (tracker) et manqué l'autre (pool).

MOYEN 3 — Un `Notify` binaire unique par tenant vs un compteur : réveil perdu

Chaque tenant partage **un seul** `Arc<Notify>` (drr.rs:161) alors que `outstanding_grants` est un **compteur**. `notify_one` de tokio **coalesce** (un seul permit stocké). Avec capacité ≥2 et deux waiters du *même* tenant, deux octrois quasi-simultanés → deux `notify_one` fusionnés en un → **un waiter réveillé, l'autre dort** jusqu'à un notify sans rapport ou son propre timeout. **Auto-réparant** (au timeout, `cancel_pending` rembobine le slot fantôme) et **jamais sur-admission** — mais un slot retenu inutilement et un rejet abusif pour une tâche qui avait pourtant un slot.

MOYEN 4 — `burst_capacity = rate` : tout appel LLM plus gros qu'une seconde de tokens est rejeté à jamais (chemin dormant)

`consume_llm_tokens` fixe `capacity = burst_capacity(rate) = rate.max(1.0)` (admission.rs:413,479), puis `TokenBucket::consume` **rejette définitivement** tout `cost > capacity` (token_bucket.rs:167 : "Even with a full bucket, this call can never succeed"). Tout appel où `tokens > max(llm_tokens_per_second, 1.0)` est impossible à satisfaire : pour un tenant à 500 tokens/s, **toute** requête LLM réaliste (des milliers de tokens) est rejetée par "exceeds bucket capacity". La rafale devrait être dimensionnée sur le coût max d'une requête, pas sur le débit/seconde. Réel — mais dans un chemin (Mode C) **non servi** (mocké), donc latent.

MOYEN 5 — La garde anti-famine est du code mort

`rounds_waited` n'est incrémenté que pour les tenants *effectivement visités* par le scan (drr.rs:662) ; comme le tenant de tête qualifie toujours et qu'`advance break`, les tenants derrière lui **ne sont jamais visités** → leur `rounds_waited` reste 0 → le seuil `rounds_waited >= max_starvation_rounds`

ne se déclenche jamais pour eux. Inoffensif *parce que* le round-robin involontaire (Constat 1) garantit déjà le service de chacun une fois par rotation — mais la machinerie anti-famine est inerte.

FAIBLE 6 — Rapportés (non entièrement re-dérivés) et nit

Suspension abortive : `acquire_op_permit` re-résout la policy à chaque op (`admission.rs:323`) ; si `policy()` erre sur tenant suspendu, les ops d'un swarm déjà admis sont refusées en plein vol, contredisant le contrat "les permits existants drainent". *Bucket figé au re-tiering* : un changement de tier ne reconfigurerait pas un bucket déjà créé. *Nit* : `Instant::now() + timeout` panique pour `Duration::MAX` (`drr.rs:417`) — inatteignable par les appelants réels (timeout défaut 5 s) ; durcir en `checked_add`.

3 Synthèse — lecture honnête

Les deux sous-systèmes sont du **vrai code sérieux et testé**, et l'audit le montre à double tranchant — exactement le registre crédible :

- **La chaîne HMAC** est cryptographiquement propre dans sa construction (HKDF par tenant, séparation de domaine tombale, tags Merkle RFC-6962), mais sa **garantie réelle est "tamper-evident pour un détenteur de clé avec tip épinglé hors-bande"** — pas trustless ni anti-opérateur : l'ancrage externe qui fermerait l'écart est stubbé. C'est un actif de **discipline** (le type impose l'honnêteté), pas un coffre-fort déployé.
- **Le DRR** préserve la **sûreté** (jamais de sur-admission) mais son **équité pondérée n'existe pas** (round-robin) et son `acquire` **fuit sous annulation** — deux défauts corrigibles et bien localisés, dans un composant de toute façon non branché en prod.

La phrase juste : *du code d'infrastructure d'agents réel et testé, dont l'audit dégage des défauts précis et corrigibles — preuve d'une capacité d'ingénierie et d'une rigueur de revue, pas d'une plateforme finie.*